JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class CharsetEncoder

java.lang.Object
    java.nio.charset.CharsetEncoder

```
public abstract class CharsetEncoder
extends Object
```

An engine that can transform a sequence of sixteen-bit Unicode characters into a sequence of bytes in a specific charset.

The input character sequence is provided in a character buffer or a series of such buffers. The output byte sequence is written to a byte buffer or a series of such buffers. An encoder should always be used by making the following sequence of method invocations, hereinafter referred to as an *encoding operation*:

1. Reset the encoder via the `reset` method, unless it has not been used before;

2. Invoke the `encode` method zero or more times, as long as additional input may be available, passing `false` for the `endOfInput` argument and filling the input buffer and flushing the output buffer between invocations;

3. Invoke the `encode` method one final time, passing `true` for the `endOfInput` argument; and then

4. Invoke the `flush` method so that the encoder can flush any internal state to the output buffer.

Each invocation of the `encode` method will encode as many characters as possible from the input buffer, writing the resulting bytes to the output buffer. The `encode` method returns when more input is required, when there is not enough room in the output buffer, or when an encoding error has occurred. In each case a `CoderResult` object is returned to describe the reason for termination. An invoker can examine this object and fill the input buffer, flush the output buffer, or attempt to recover from an encoding error, as appropriate, and try again.

There are two general types of encoding errors. If the input character sequence is not a legal sixteen-bit Unicode sequence then the input is considered *malformed*. If the input character sequence is legal but cannot be mapped to a valid byte sequence in the given charset then an *unmappable character* has been encountered.

How an encoding error is handled depends upon the action requested for that type of error, which is described by an instance of the `CodingErrorAction` class. The possible error actions are to ignore the erroneous input, report the

error to the invoker via the returned `CoderResult` object, or `replace` the erroneous input with the current value of the replacement byte array. The replacement is initially set to the encoder's default replacement, which often (but not always) has the initial value `{ (byte)'?' }`; its value may be changed via the `replaceWith` method.

The default action for malformed-input and unmappable-character errors is to `report` them. The malformed-input error action may be changed via the `onMalformedInput` method; the unmappable-character action may be changed via the `onUnmappableCharacter` method.

This class is designed to handle many of the details of the encoding process, including the implementation of error actions. An encoder for a specific charset, which is a concrete subclass of this class, need only implement the abstract `encodeLoop` method, which encapsulates the basic encoding loop. A subclass that maintains internal state should, additionally, override the `implFlush` and `implReset` methods.

Instances of this class are not safe for use by multiple concurrent threads.

Since:

1.4

See Also:

`ByteBuffer`, `CharBuffer`, `Charset`, `CharsetDecoder`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`CharsetEncoder (Charset cs, float averageBytesPerCha float maxBytesPerChar)`	Initializes a new encoder.
protected	`CharsetEncoder (Charset cs, float averageBytesPerCha float maxBytesPerChar, byte[] replacement)`	Initializes a new encoder.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
final float	**averageBytesPerChar**()	Returns the average number of bytes that will be produced for each character of input.
boolean	**canEncode**(char c)	Tells whether or not this encoder can encode the given character.
boolean	**canEncode** (CharSequence cs)	Tells whether or not this encoder can encode the given character sequence.
final Charset	**charset**()	Returns the charset that created this encoder.
final ByteBuffer	**encode**(CharBuffer in)	Convenience method that encodes the remaining content of a single input character buffer into a newly-allocated byte buffer.
final CoderResult	**encode**(CharBuffer in, ByteBuffer out, boolean endOfInput)	Encodes as many characters as possible from the given input buffer, writing the results to the given output buffer.
protected abstract CoderResult	**encodeLoop** (CharBuffer in, ByteBuffer out)	Encodes one or more characters into one or more bytes.
final CoderResult	**flush**(ByteBuffer out)	Flushes this encoder.
protected CoderResult	**implFlush** (ByteBuffer out)	Flushes this encoder.
protected void	**implOnMalformedInput** (CodingErrorAction newAc	Reports a change to this encoder's malformed-input action.
protected void		

	implOnUnmappableCharacte (CodingErrorAction newAc	Reports a change to this encoder's unmappable-character action.
protected void	**implReplaceWith** (byte[] newReplacement)	Reports a change to this encoder's replacement value.
protected void	**implReset**()	Resets this encoder, clearing any charset-specific internal state.
boolean	**isLegalReplacement** (byte[] repl)	Tells whether or not the given byte array is a legal replacement value for this encoder.
CodingErrorAction	**malformedInputAction**()	Returns this encoder's current action for malformed-input errors.
final float	**maxBytesPerChar**()	Returns the maximum number of bytes that will be produced for each character of input.
final CharsetEncoder	**onMalformedInput** (CodingErrorAction newAc	Changes this encoder's action for malformed-input errors.
final CharsetEncoder	**onUnmappableCharacter** (CodingErrorAction newAc	Changes this encoder's action for unmappable-character errors.
final byte[]	**replacement**()	Returns this encoder's replacement value.
final CharsetEncoder	**replaceWith** (byte[] newReplacement)	Changes this encoder's replacement value.
final CharsetEncoder	**reset**()	Resets this encoder, clearing any internal state.
CodingErrorAction	**unmappableCharacterActio**	Returns this encoder's current action for

	unmappable-character errors.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

CharsetEncoder

```
protected CharsetEncoder(Charset cs,
                         float averageBytesPerChar,
                         float maxBytesPerChar,
                         byte[] replacement)
```

Initializes a new encoder. The new encoder will have the given bytes-per-char and replacement values.

Parameters:

`cs` - The charset that created this encoder

`averageBytesPerChar` - A positive float value indicating the expected number of bytes that will be produced for each input character

`maxBytesPerChar` - A positive float value indicating the maximum number of bytes that will be produced for each input character

`replacement` - The initial replacement; must not be `null`, must have non-zero length, must not be longer than maxBytesPerChar, and must be legal

Throws:

`IllegalArgumentException` - If the preconditions on the parameters do not hold

CharsetEncoder

```
protected CharsetEncoder(Charset cs,
                         float averageBytesPerChar,
                         float maxBytesPerChar)
```

Initializes a new encoder. The new encoder will have the given bytes-per-char values and its replacement will be the byte array `{ (byte)'?' }`.

Parameters:

`cs` - The charset that created this encoder

`averageBytesPerChar` - A positive float value indicating the expected number of bytes that will be produced for each input character

`maxBytesPerChar` - A positive float value indicating the maximum number of bytes that will be produced for each input character

Throws:

`IllegalArgumentException` - If the preconditions on the parameters do not hold

Method Details

charset

`public final Charset charset()`

Returns the charset that created this encoder.

Returns:

This encoder's charset

replacement

`public final byte[] replacement()`

Returns this encoder's replacement value.

Returns:

This encoder's current replacement, which is never `null` and is never empty

replaceWith

`public final CharsetEncoder replaceWith(byte[] newReplacement)`

Changes this encoder's replacement value.

This method invokes the `implReplaceWith` method, passing the new replacement, after checking that the new replacement is acceptable.

Parameters:

`newReplacement` - The new replacement; must not be `null`, must have non-zero length, must not be longer than the value returned by the `maxBytesPerChar` method, and must be `legal`

Returns:

This encoder

Throws:

`IllegalArgumentException` - If the preconditions on the parameter do not hold

implReplaceWith

```
protected void implReplaceWith(byte[] newReplacement)
```

Reports a change to this encoder's replacement value.

The default implementation of this method does nothing. This method should be overridden by encoders that require notification of changes to the replacement.

Parameters:

`newReplacement` - The replacement value

isLegalReplacement

```
public boolean isLegalReplacement(byte[] repl)
```

Tells whether or not the given byte array is a legal replacement value for this encoder.

A replacement is legal if, and only if, it is a legal sequence of bytes in this encoder's charset; that is, it must be possible to decode the replacement into one or more sixteen-bit Unicode characters.

The default implementation of this method is not very efficient; it should generally be overridden to improve performance.

Parameters:

`repl` - The byte array to be tested

Returns:

`true` if, and only if, the given byte array is a legal replacement value for this encoder

malformedInputAction

```
public CodingErrorAction malformedInputAction()
```

Returns this encoder's current action for malformed-input errors.

Returns:

The current malformed-input action, which is never `null`

onMalformedInput

```
public final CharsetEncoder onMalformedInput
(CodingErrorAction newAction)
```

Changes this encoder's action for malformed-input errors.

This method invokes the `implOnMalformedInput` method, passing the new action.

Parameters:

`newAction` - The new action; must not be `null`

Returns:

This encoder

Throws:

`IllegalArgumentException` - If the precondition on the parameter does not hold

implOnMalformedInput

```
protected void implOnMalformedInput(CodingErrorAction newAction)
```

Reports a change to this encoder's malformed-input action.

The default implementation of this method does nothing. This method should be overridden by encoders that require notification of changes to the malformed-input action.

Parameters:

`newAction` - The new action

unmappableCharacterAction

```
public CodingErrorAction unmappableCharacterAction()
```

Returns this encoder's current action for unmappable-character errors.

Returns:

The current unmappable-character action, which is never `null`

onUnmappableCharacter

```
public final CharsetEncoder onUnmappableCharacter
(CodingErrorAction newAction)
```

Changes this encoder's action for unmappable-character errors.

This method invokes the `implOnUnmappableCharacter` method, passing the new action.

Parameters:

`newAction` - The new action; must not be `null`

Returns:

This encoder

Throws:

`IllegalArgumentException` - If the precondition on the parameter does not hold

implOnUnmappableCharacter

```
protected void implOnUnmappableCharacter(CodingErrorAction newAction)
```

Reports a change to this encoder's unmappable-character action.

The default implementation of this method does nothing. This method should be overridden by encoders that require notification of changes to the unmappable-character action.

Parameters:

`newAction` - The new action

averageBytesPerChar

```
public final float averageBytesPerChar()
```

Returns the average number of bytes that will be produced for each character of input. This heuristic value may be used to estimate the size of the output buffer required for a given input sequence.

Returns:

The average number of bytes produced per character of input

maxBytesPerChar

```
public final float maxBytesPerChar()
```

Returns the maximum number of bytes that will be produced for each character of input. This value may be used to compute the worst-case size of the output buffer required for a given input sequence. This value accounts for any necessary content-independent prefix or suffix bytes, such as byte-order marks.

Returns:

The maximum number of bytes that will be produced per character of input

encode

```
public final CoderResult encode(CharBuffer in,
                                ByteBuffer out,
                                boolean endOfInput)
```

Encodes as many characters as possible from the given input buffer, writing the results to the given output buffer.

The buffers are read from, and written to, starting at their current positions. At most `in.remaining()` characters will be read and at most `out.remaining()` bytes will be written. The buffers' positions will be advanced to reflect the characters read and the bytes written, but their marks and limits will not be modified.

In addition to reading characters from the input buffer and writing bytes to the output buffer, this method returns a `CoderResult` object to describe its reason for termination:

- `CoderResult.UNDERFLOW` indicates that as much of the input buffer as possible has been encoded. If there is no further input then the invoker can proceed to the next step of the encoding operation. Otherwise this method should be invoked again with further input.
- `CoderResult.OVERFLOW` indicates that there is insufficient space in the output buffer to encode any more characters. This method should be invoked again with an output buffer that has more remaining bytes. This is typically done by draining any encoded bytes from the output buffer.
- A malformed-input result indicates that a malformed-input error has been detected. The malformed characters begin at the input buffer's (possibly incremented) position; the number of malformed characters may be determined by invoking the result object's `length` method. This case applies only if the malformed action of this encoder is `CodingErrorAction.REPORT`; otherwise the malformed input will be ignored or replaced, as requested.
- An unmappable-character result indicates that an unmappable-character error has been detected. The characters that encode the unmappable character begin at the input buffer's (possibly incremented) position; the number of such characters may be determined by invoking the result object's `length` method. This case applies only if the unmappable action of this encoder is `CodingErrorAction.REPORT`; otherwise the unmappable character will be ignored or replaced, as requested.

In any case, if this method is to be reinvoked in the same encoding operation then care should be taken to preserve any characters remaining in the input buffer so that they are available to the next invocation.

The `endOfInput` parameter advises this method as to whether the invoker can provide further input beyond that contained in the given input buffer. If there is a possibility of providing additional input then the invoker should pass `false` for this parameter; if there is no possibility of providing further input then the invoker should pass `true`. It is not erroneous, and in fact it is quite common, to pass `false` in one invocation and later discover that no further input was actually available. It is critical, however, that the final invocation of this method in a sequence of invocations always pass `true` so that any remaining unencoded input will be treated as being malformed.

This method works by invoking the `encodeLoop` method, interpreting its results, handling error conditions, and reinvoking it as necessary.

Parameters:

`in` - The input character buffer

`out` - The output byte buffer

`endOfInput` - `true` if, and only if, the invoker can provide no additional input characters beyond those in the given buffer

Returns:

A coder-result object describing the reason for termination

Throws:

`IllegalStateException` - If an encoding operation is already in progress and the previous step was an invocation neither of the `reset` method, nor of this method with a value of `false` for the `endOfInput` parameter, nor of this method with a value of `true` for the `endOfInput` parameter but a return value indicating an incomplete encoding operation

`CoderMalfunctionError` - If an invocation of the encodeLoop method threw an unexpected exception

flush

```
public final CoderResult flush(ByteBuffer out)
```

Flushes this encoder.

Some encoders maintain internal state and may need to write some final bytes to the output buffer once the overall input sequence has been read.

Any additional output is written to the output buffer beginning at its current position. At most `out.remaining()` bytes will be written. The buffer's position will be advanced appropriately, but its mark and limit will not be modified.

If this method completes successfully then it returns `CoderResult.UNDERFLOW`. If there is insufficient room in the output buffer then it returns `CoderResult.OVERFLOW`. If this happens then this method must be invoked again, with an output buffer that has more room, in order to complete the current encoding operation.

If this encoder has already been flushed then invoking this method has no effect.

This method invokes the `implFlush` method to perform the actual flushing operation.

Parameters:

`out` - The output byte buffer

Returns:

A coder-result object, either `CoderResult.UNDERFLOW` or `CoderResult.OVERFLOW`

Throws:

`IllegalStateException` - If the previous step of the current encoding operation was an invocation neither of the `flush` method nor of the three-argument `encode` method with a value of `true` for the `endOfInput` parameter

implFlush

```
protected CoderResult implFlush(ByteBuffer out)
```

Flushes this encoder.

The default implementation of this method does nothing, and always returns `CoderResult.UNDERFLOW`. This method should be overridden by encoders that may need to write final bytes to the output buffer once the entire input sequence has been read.

Parameters:

`out` - The output byte buffer

Returns:

A coder-result object, either `CoderResult.UNDERFLOW` or `CoderResult.OVERFLOW`

reset

```
public final CharsetEncoder reset()
```

Resets this encoder, clearing any internal state.

This method resets charset-independent state and also invokes the `implReset` method in order to perform any charset-specific reset actions.

Returns:

This encoder

implReset

```
protected void implReset()
```

Resets this encoder, clearing any charset-specific internal state.

The default implementation of this method does nothing. This method should be overridden by encoders that maintain internal state.

encodeLoop

```
protected abstract CoderResult encodeLoop(CharBuffer in,
                                          ByteBuffer out)
```

Encodes one or more characters into one or more bytes.

This method encapsulates the basic encoding loop, encoding as many characters as possible until it either runs out of input, runs out of room in the output buffer, or encounters an encoding error. This method is invoked by the `encode` method, which handles result interpretation and error recovery.

The buffers are read from, and written to, starting at their current positions. At most `in.remaining()` characters will be read, and at most `out.remaining()` bytes will be written. The buffers' positions will be advanced to reflect the characters read and the bytes written, but their marks and limits will not be modified.

This method returns a `CoderResult` object to describe its reason for termination, in the same manner as the `encode` method. Most implementations of this method will handle encoding errors by returning an appropriate result object for interpretation by the `encode` method. An optimized implementation may instead examine the relevant error action and implement that action itself.

An implementation of this method may perform arbitrary lookahead by returning `CoderResult.UNDERFLOW` until it receives sufficient input.

Parameters:

`in` - The input character buffer

`out` - The output byte buffer

Returns:

A coder-result object describing the reason for termination

encode

```
public final ByteBuffer encode(CharBuffer in)
                        throws CharacterCodingException
```

Convenience method that encodes the remaining content of a single input character buffer into a newly-allocated byte buffer.

This method implements an entire encoding operation; that is, it resets this encoder, then it encodes the characters in the given character buffer, and finally it flushes this encoder. This method should therefore not be invoked if an encoding operation is already in progress.

Parameters:

`in` - The input character buffer

Returns:

A newly-allocated byte buffer containing the result of the encoding operation. The buffer's position will be zero and its limit will follow the last byte written.

Throws:

`IllegalStateException` - If an encoding operation is already in progress

`MalformedInputException` - If the character sequence starting at the input buffer's current position is not a legal sixteen-bit Unicode sequence and the current malformed-input action is `CodingErrorAction.REPORT`

`UnmappableCharacterException` - If the character sequence starting at the input buffer's current position cannot be mapped to an equivalent byte sequence and the current unmappable-character action is `CodingErrorAction.REPORT`

`CharacterCodingException` - `MalformedInputException` if the character sequence starting at the input buffer's current position is not a legal sixteen-bit Unicode sequence and the current malformed-input action is `CodingErrorAction.REPORT`; `UnmappableCharacterException` if the character sequence starting at the input buffer's current position cannot be mapped to an equivalent byte sequence and the current unmappable-character action is `CodingErrorAction.REPORT`

`OutOfMemoryError` - If the output byte buffer for the requested size of the input character buffer cannot be allocated

canEncode

```
public boolean canEncode(char c)
```

Tells whether or not this encoder can encode the given character.

This method returns `false` if the given character is a surrogate character; such characters can be interpreted only when they are members of a pair consisting of a high surrogate followed by a low surrogate. The `canEncode(CharSequence)` method may be used to test whether or not a character sequence can be encoded.

This method may modify this encoder's state; it should therefore not be invoked if an encoding operation is already in progress.

The default implementation of this method is not very efficient; it should generally be overridden to improve performance.

Parameters:

`c` - The given character

Returns:

`true` if, and only if, this encoder can encode the given character

Throws:

`IllegalStateException` - If an encoding operation is already in progress

canEncode

```
public boolean canEncode(CharSequence cs)
```

Tells whether or not this encoder can encode the given character sequence.

If this method returns `false` for a particular character sequence then more information about why the sequence cannot be encoded may be obtained by performing a full encoding operation.

This method may modify this encoder's state; it should therefore not be invoked if an encoding operation is already in progress.

The default implementation of this method is not very efficient; it should generally be overridden to improve performance.

Parameters:

`cs` - The given character sequence

Returns:

`true` if, and only if, this encoder can encode the given character without throwing any exceptions and without performing any replacements

Throws:

`IllegalStateException` - If an encoding operation is already in progress

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.